SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

Project Title: ***LHDN Fraud Detective Agent***

Prepared By: ***Team CacheMe (Domain 1 – Group ID: 103)***

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. *Introduction:*

The LHDN Fraud Detective Agent is a workflow prototype that combines document understanding, operator correction, database-backed compliance checks, and ledger actions in a single verification loop. It is designed for the UMHackathon 2026 Domain 1 brief, where GLM must act as the central reasoning engine rather than a peripheral chatbox layer.

1.1. Purpose:

This document translates the repository into a technical system view. It explains how the frontend, backed, GLM layer, database, and decision logic work together, what the data model looks like, which features belong to the hackathon MVP, and which deployment controls remain recommendations rather than implemented capabilities.

Key elements covered in this document:

1. *System architecture and current deployment shape*
2. *End-to-end workflow and sequence logic*
3. *GLM integration and prompt construction*
4. *Database schema and core relations*
5. *Functional scope, non-functional requirements, and limitations*
6. *Operational assumptions, external dependencies, and recommended scale-up steps*

1.2. Background:

***Operational Baseline**:*
The practical baseline is a manual invoice-review process where users inspect incoming files, identify supplier and invoice details, compare data to internal records, and decide how to handle the transaction. The repository does not contain a previous software release; instead, this prototype replaces that manual baseline with a hybrid AI-plus rule workflow.

1. *Previous Version:*
   No earlier application version is present in the codebase. The meaningful 'before state' is a human-only review path without unified extraction, automated duplicate detection, or retail consolidation support.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. *Changes in Major Architectural Components:*
   The prototype introduces a chat-style upload UI, a FastAPI orchestration layer, a GLM extraction layer, SQLite-backed verification rules, a B2B certificate path, and a B2C consolidation path. These are the core architectural shifts from the manual baseline.

3. *New Capabilities Introduced:*
   structured JSON extraction from unstructured documents, guided missing-field completion, supplier and contract cross-checking, persistent workflow ledgers, and command-driven month-end aggregation.

1.3. Target Stakeholder

Stakeholder	***Role in Workflow***	***Expectation***
Finance / Accounts Payable Operator	*Uploads and reviews enterprise invoices*	*Fast extraction, clear validation output, printable approval evidence*
Procurement / Compliance Reviewer	*Investigates warnings and fraud flags*	*Supplier identity checks, duplicate detection, contract-price visibility*
Retail Operations Administrator	*Logs and consolidates cash receipts*	*Reliable monthly aggregation with minimal manual work*
QA	*Evaluate reliability and workflow fit*	*Visible traceability from AI extraction to system action*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Current Repository Reality
System	*Web-based operator console backed by FastAPI and SQLite*
Architecture	*Monolithic application service with one external model API*
Frontend	*Static HTML / CSS/ JavaScript page with browser-native speech recognition hooks*
Backend	*Single FastAPI app in main.py*
Persistence	*SQLite database with 4 vendors, 5 products, 5 contract rules, 3 seeded processed invoices, plus operational ledgers for retail and approvals*
AI Layer	*ILMU chat-completions endpoint using model ilmu-glm-5.1*

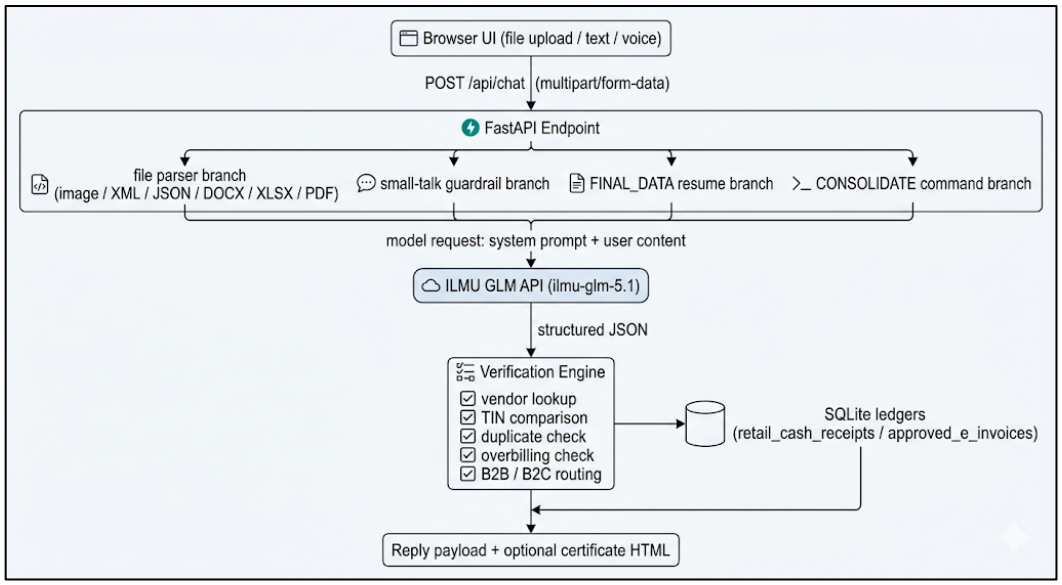
The current implementation is a single-node client-server prototype. The browser serves a static operator console from templates/index.html over port 8080, while a FastAPI backend on port 8000 exposes one workflow endpoint at /api/chat. The backend preprocesses uploaded documents, invokes the GLM extraction service, executes deterministic verification logic, writes selected workflow state to SQLite, and returns a response containing action metadata and optional certificate HTML.

2.1.2 LLM as Service Layer

- The GLM layer is not optional decoration. It is responsible for converting raw documents into the structured JSON object used by every downstream rule.
- Without the GLM response, the backend cannot populate supplier details, invoice metadata, financial totals, or line items from unstructured inputs.
- The backend treats the model output as a typed interface: it requires a JSON object with *supplier_details*, *buyer_details*, *invoice_metadata*, *financials*, and *line_items*.
- The deterministic Python layer then performs business verification, routing, and persistence. This separation makes the workflow both explainable and extensible.

© UMHackathon 2026

2.1.3 Dependency Diagram

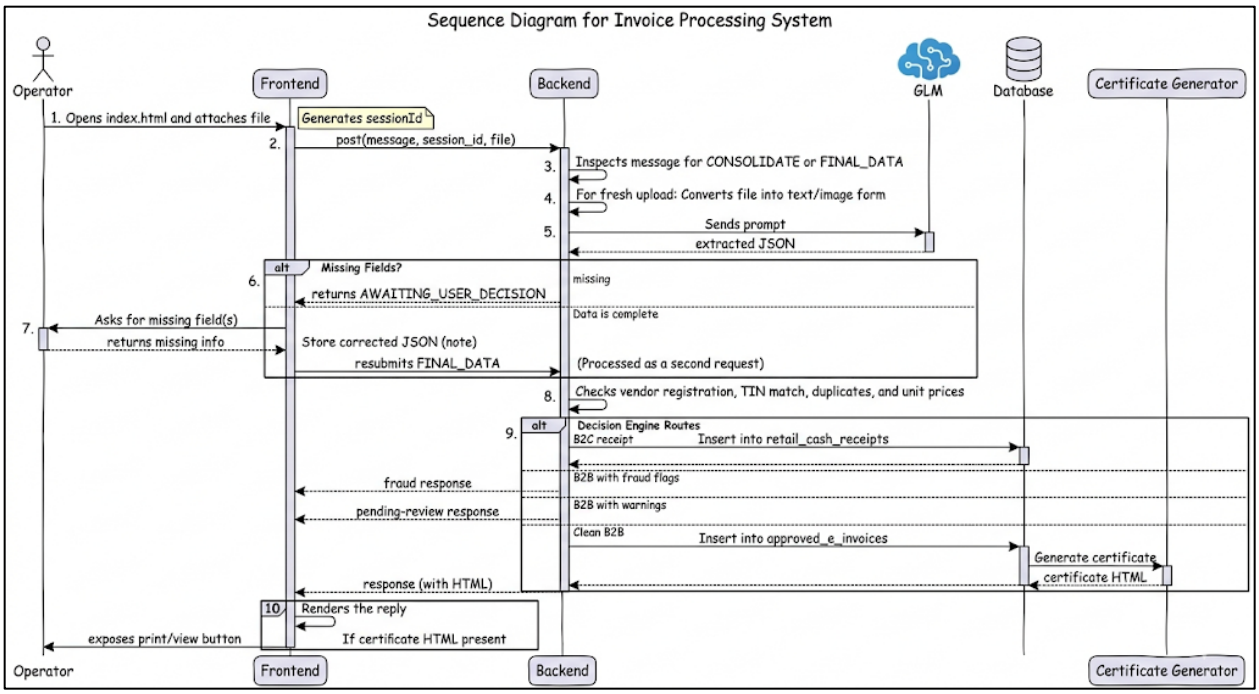


Prompt Construction and Context Window:
The system prompt hardcodes OCR and extraction rules oriented toward Malaysian invoices. Image uploads receive additional visual layout instructions and a base64 image payload. Text-derived documents receive the operator message plus a raw-document block appended to the same request. The current code does not implement chunking or explicit token-window enforcement, so the system inherits the context limitations of the external model endpoint.

Response Parsing and Downstream Handoff:
The backend searches for the first JSON object in the model response and loads it into Python. If parsing fails, the request ends in an error state. If the JSON exists but key fields are missing, the frontend stores partial_data and pendingFieldsToFix in the browser, then resubmits the corrected payload to the backend through the FINAL_DATA mechanism.

2.1.4 Sequence Diagram

© UMHackathon 2026



2.2. *Technological Stack*

Layer	*Technology*	*Current Use in Repo*
Frontend	*HTML5 / CSS / JavaScript*	*Single chat-style page for upload, voice input, response rendering, and certificate printing*
Backend	*FastAPI*	*One POST endpoint that orchestrates parsing, model interaction, verification, and response generation*
Database	*SQLite*	*Local persistence for vendors, contracts, processed invoices, retail receipts, and approved invoice ledger*
AI Service	*ILMU endpoint / model ilmu-glm-5.1*	*Structured extraction from image and text-derived invoice content*
Document Parsing	*Python-docx, pandas, PyPDF2*	*Reads DOCX, XLSX, and PDF contents before the GLM call*

© UMHackathon 2026

Browser Capability	*SpeechRecognition / webkitSpeechRecognition*	*Optimal microphone-based instruction entry in supported browsers*

2.3. Key Data Flows

1. ***Ingestion flow:***
   A user uploads a document or enters a command from the browser UI.

2. ***Preprocessing flow:***
   The backend converts the artifact into either base64 image form or extracted text form.

3. ***Extraction flow:***
   The GLM model returns structured invoice JSON.

4. ***Validation flow:***
   The backend checks supplier, TIN, duplicate history, and contract pricing.

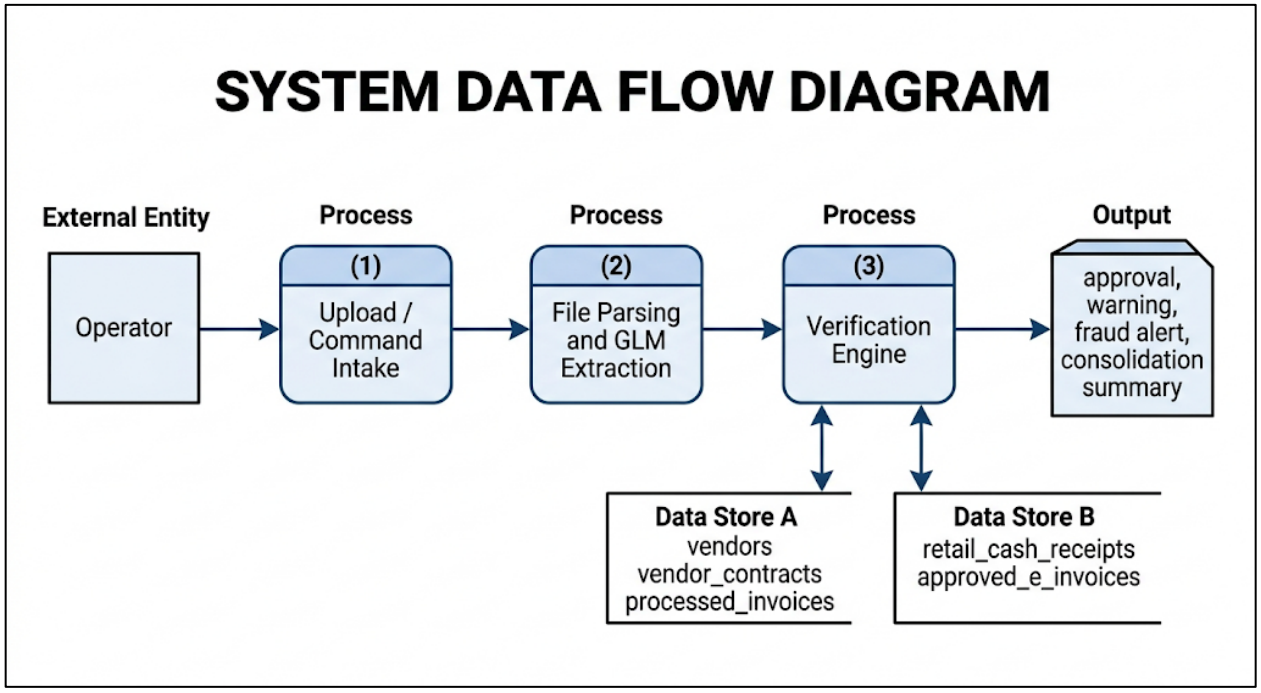
5. ***Routing flow:***
   The decision engine chooses B2C ledgering, fraud, warning, or approval.

6. ***Persistence flow:***
   Selected outcomes are inserted into retail_cash_receipts or approved_e_invoices; historical duplicate detection reads processed_invoices.

7. ***Presentation flow:***
   The frontend renders result text and exposes printable certificate content where relevant.

© UMHackathon 2026

2.3.1.1. Data Flow Diagram (DFD)



2.3.2. Normalized Database Schema

Table	***Important Fields***	***Purpose***	***3NF / Notes***
vendors	*vendor_id, company_name, ssm_number, tax_id, status*	*Registered suppliers used for identity checks*	*One row per supplier; no repeating groups*
products	*sku, product_name, category*	*Catalog of items used for contract matching*	*Lookup table keyed by SKU*
vendor_contracts	*contract_id, vendor_id, sku, agreed_unit_price, valid_until*	*Contract-price bridge between vendor and SKU*	*Resolves vendor-to-product pricing without duplication*
processed_invoices	*invoice_id, invoice_numberm vendor_id, total_amount, processed_date*	*Historical invoice ledger for duplicate detection*	*Unique(invoice_number, vendor_id) enforces baseline duplicate integrity*

© UMHackathon 2026

Email: umhackathon@um.edu.my

retail_cash_receipts	*receipt_id, receipt_number, transaction_date, total_amount, status*	*B2C ledger awaiting or completing consolidation*	*No unique constraint on receipt_number in current schema*
approved_e_invoices	*approval_id, invoice_number, supplier_ssm, buyer_name, grand_total, lhdn_uuid, approved_time*	*Approved invoice and consolidated batch ledger*	*Separate approval ledger; not currently used for duplicate lookup*

3. Functional Requirements & Scope

ID	***Functional Requirement***	***Repository Evidence***
FR-01	*Accept multi-format uploads*	*main.py parses image, XML, JSON, DOCX, XLSX, and PDF*
FR-02	*Extract structured invoice JSON*	*system prompt + GLM call in analyze_document_with_ai*
FR-03	*Recover from missing critical fields*	*AWAITING_USER_DECISION flow in backend and frontend*
FR-04	*Verify supplier and TIN against master data*	*SQLite vendor lookup and TIN comparison*
FR-05	*Detect duplicates and overbilling*	*processed_invoices and vendor_contracts checks*
FR-06	*Classify B2B vs B2C and persist outcome*	*buyer-name based routing in main.py*
FR-07	*Consolidate B2C receipts via command*	*CONSOLIDATE branch*
FR-08	*Generate approval certificate for clean B2B invoices*	*certificate HTML returned in response*

© UMHackathon 2026

3.1. Minimum Viable Product:

#	MVP Feature	Why it Matters for Demo
1	Upload any supported invoice artifact	Shows unstructured-input handling immediately
2	GLM extraction into fixed JSON	Demonstrates AI as the workflow reasoning layer
3	Guided missing-field repair	Demonstrates resilience to ambiguous or poor-quality inputs
4	Deterministic fraud/compliance checks	Shows that AI output is acted upon rather than merely displayed
5	B2B approval and B2C consolidation split	Demonstrates workflow branching and stateful ledgers

3.2. Non-Functional Requirements (NFRs)

Quality	Requirement	Current Implementation / Observation
Reliability	System should fail visibly, not silently	Timeout retry, JSON parse guard, human-correction path, warning/fraud states
Maintainability	Business logic should remain understandable	Prototype is readable but concentrated in one main.py file
Security	Secrets and access should be controlled	Current repo still contains a hardcoded API key and wildcard CORS; requires hardening
Cost Efficiency	AI usage should be targeted	Single extraction call per document, then deterministic Python checks
Performance	Demo flow should complete in a reasonable operator window	Synchronous end-to-end flow with 120-second model timeout and no queue

© UMHackathon 2026

Usability	*Operator should understand what the system wants next*	*Clear status headers, guided missing fields, certificate button, and command-based consolidation*

3.3. Out of Scope / Future Enhancements

- *Real LHDN connector with signed API integration instead of local UUID simulation.*
- *Authentication, audit identities, and role-based approval chains.*
- *Asynchronous job queue for long-running extraction and bulk ingestion.*
- *Larger master-data ingestion from ERP or vendor onboarding systems.*
- *Unified duplicate-check ledger so new approvals immediately affect duplicate screening.*
- *Cloud-native deployment, observability, and secret management.*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

The current repository is suitable for a local demo but not yet a production rollout. It already demonstrates the essential workflow integration required by the domain. The next engineering milestone is not more feature breadth; it is hardening the existing path through automated tests, larger reference data, and production-safe configuration.

4.1.1. Grayscale Rollout & A/B Testing Recommendation:

Not implemented in the current repository. For the next phase, the safest rollout would start with internal finance operators using only non-production documents, then gradually expand coverage by document type and vendor cohort. UI and prompt changes should be compared using documented extraction accuracy and review-time reduction metrics.

4.1.2. Emergency Rollback & Golden Release Recommendation:

Not implemented as a pipeline. In the current prototype, rollback is effectively code revert plus SQLite restore. For a real deployment, the golden path should include versioned prompt templates, database backups, and one-click rollback to the last known stable model/prompt combination.

© UMHackathon 2026

4.1.3. Priority Matrix:

Priority	Trigger Condition	Required Action
P1 Critical	Hardcoded secret exposure, broken approval path, or corrupted ledger writes	Freeze deployment, rotate secrets, restore last clean release
P2 High	Spike in model timeouts or invalid JSON responses	Switch to graceful degradation and investigate prompt or provider issues
P3 Medium	High warning volume from unknown vendors or SKUs	Review reference-data coverage and refine onboarding process
P4 Low	Minor UI defects with correct backend output	Patch in next maintenance cycle

4.2. Monitoring:

Monitoring Item	Why It Matters	Current Status
GLM timeout rate	Core extraction dependency can stall the workflow	Recommended metric; not instrumented
Invalid JSON rate	Measures extraction stability against the schema contract	Recommended metric; not instrumented
Warning-to-approval ratio	Signals master-data coverage and operator burdern	Recommended metric; not instrumented
Duplicate and overbilling hits	Measures fraud signal capture	Observable through response, not aggreagated
Retail consolidation backlog	Shows whether B2C receipts are being cleared in time	Queryable from SQLite

4.3. Assumptions

a. Operators are reviewing Malaysian invoice-like documents rather than arbitrary files.
b. Internet access is available when GLM extraction is required.
c. Seed data remains curated enough to support the intended demo path.

© UMHackathon 2026

4.4. External Dependencies:

a. *External ILMU model endpoint.*
b. *Python document-parsing libraries and local SQLite database file.*
c. *Modern browser support for the upload UI and optional voice input.*

5. Project Management & Team Contributions

5.1. Project Timeline:

This project timeline is presented as the team's planned development schedule for the UMHackathon 2026 preliminary round, aligned with the official organiser window from 16 April 2026, 21:00:00 to 26 April 2026, 07:59:59. The purpose of this timeline is to show how the team structured planning, implementation, testing, and submission work to ensure that the prototype, documentation, and presentation materials could be completed within the allowed period. Task ownership reflects the team's confirmed role distribution, while major workflow decisions were made jointly by all five team members.

Day	**Planned Focus**	**Key Activities**	**Justification**
Day 0	*Domain understanding and kickoff*	*Attend domain reveal, analyse problem statement, review judging criteria, align on target user flow and solution direction*	*The team needed a common understanding of the assigned problem before any technical or documentation work could begin.*
Day 1	*Project planning and architecture scoping*	*Define system workflow, identify major modules, decide stack, assign core responsibilities across PRD, SAD, implementation, QATD, slide deck, and video preparation*	*Early role clarity reduces duplicated effort and helps the team move in parallel during a short hackathon window.*
Day 2	*Data model and prototype foundation*	*Set up repository structure, database schema, baseline backend flow, and initial UI skeleton; confirm input/output expectations for invoice processing*	*Establishing the technical foundation early is necessary so later implementation and testing can proceed against a stable base.*
Day 3	*Core processing workflow*	*Implement document upload, parsing flow, model interaction path, and initial response handling*	*This is the core value path of the system, so it should be built early to validate technical feasibility.*

© UMHackathon 2026

Day 4	*Fraud and compliance logic*	*Implement vendor validation, duplicate checking, invoice verification logic, and approval / warning / fraud branching*	*These checks directly support the problem statement and judging expectations around practical system intelligence.*
Day 5	*Retail and exception workflow*	*Implement B2C receipt handling, consolidation flow, and user correction path for incomplete extracted fields*	*Supporting both standard and edge workflows improves completeness and demonstrates realistic operational thinking.*
Day 6	*Integration refinement and internal review*	*Connect frontend and backend flows fully, review database writes, improve prompt / extraction handling, and validate end-to-end logic*	*Midway integration helps surface design gaps before the team becomes locked into documentation and presentation work.*
Day 7	*Documentation drafting*	*PRD drafting by Eldeena and Kong Yi Xuan; SAD drafting by Do Wai Lung; QATD drafting by Fernando and Beckham*	*Documentation should be developed alongside the prototype so that claims remain aligned with the actual implementation.*
Day 8	*QA pass and evidence collection*	*Run functional checks, verify happy-path and edge-path behaviour, inspect outputs, capture screenshots / evidence, and identify remaining gaps*	*A dedicated QA stage is required to ensure that the final submission is coherent, evidence-backed, and less prone to contradiction.*
Day 9	*Presentation and demo preparation*	*Prepare pitch deck, structure pitch storyline, record demo flow, refine narrative around problem, solution, architecture, and impact*	*The judging rubric rewards clarity and persuasiveness, so technical work must be translated into a strong presentation narrative.*
Day 10	*Final review and submission readiness*	*Final code review, document consistency check, repository cleanup, verification of links/files, and submission packaging by team leader*	*Because the organiser allows only one irreversible submission, the final stage must focus on accuracy, consistency, and submission control.*

© UMHackathon 2026

5.2. Team Members Role:

Team Member	Confirmed Contribution Area
Do Wai Lung	Team leader; final code review; SAD ownership; pitching-video presenter; shared workflow design
Fernando Lim Jia Yee	Code implementation; QATD ownership; demo video preparation; shared workflow design
Beckham Wong Yao Kang	Code implementation; QATD ownership; demo video preparation; shared workflow design
Eldeena Lim Huey Yinn	PRD ownership; pitch deck preparation; shared workflow design
Kong Yi Xuan	PRD ownership; pitch deck preparation; shared workflow design

5.3. Recommendations:

- *Refactor main.py into parsing, model-client, verification, and persistence modules to improve testability.*
- *Move the API key out of source code and restrict CORS before any external demo deployment.*
- *Add application-level file-size and token-budget guards with chunking or rejection behavior.*
- *Unify invoice history so newly approved B2B invoices also participate in duplicate checks.*
- *Expand the live vendor and contract reference dataset beyond the minimal SQLite seed.*
- *Add a real LHDN integration adapter only after the local audit trail is stable and test-covered.*

© UMHackathon 2026